

Stats-506 Problem Set #4

GitHub Repository: <https://github.com/zkl2002/Stats-506/>

Zhekai Liu

```
library(nzelect)
```

```
Warning:  'nzelect' R 4.4.3
```

```
library(tidyverse)
```

```
Warning:  'tidyverse' R 4.4.3
```

```
Warning:  'ggplot2' R 4.4.3
```

```
Warning:  'readr' R 4.4.3
```

```
Warning:  'forcats' R 4.4.3
```

```
Warning:  'lubridate' R 4.4.3
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
v dplyr      1.1.4      v readr      2.1.5
```

```
v forcats    1.0.0      v stringr    1.5.1
```

```
v ggplot2    4.0.0      v tibble     3.2.1
```

```
v lubridate  1.9.4      v tidyr      1.3.1
```

```
v purrr      1.0.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(ggplot2)
library(scales)
```

Warning: 'scales' R 4.4.3

'scales'

The following object is masked from 'package:purrr':

discard

The following object is masked from 'package:readr':

col_factor

```
library(slider)
```

Warning: 'slider' R 4.4.3

Problem 1 - Tidyverse: New Zealand

(a).

```
nzge %>%
  count(election_year, voting_type, wt = votes, name = "vote_count") %>%
  arrange(desc(vote_count))
```

	election_year	voting_type	vote_count
1	2014	Party	2416479
2	2014	Candidate	2375493
3	2008	Party	2356536
4	2008	Candidate	2325598
5	2005	Party	2286190
6	2005	Candidate	2260670
7	2011	Party	2257336
8	2011	Candidate	2225766
9	2002	Party	2040248
10	2002	Candidate	2022115

(b).

```
nzge %>%
  filter(election_year == 2014, voting_type == "Candidate") %>%
  count(party, wt = votes, name = "votes") %>%
  mutate(percent = votes / sum(votes) * 100) %>%
  arrange(desc(percent))
```

	party	votes	percent
1	National Party	1081787	45.539473280
2	Labour Party	801287	33.731398072
3	Green Party	165718	6.976151898
4	Conservative Party	81075	3.412975749
5	New Zealand First Party	73384	3.089211376
6	Maori Party	42108	1.772600467
7	MANA Movement	32333	1.361106937
8	Informal Candidate Votes	27886	1.173903691
9	ACT New Zealand	27778	1.169357266
10	United Future	14722	0.619745038
11	Aotearoa Legalise Cannabis Party	4936	0.207788446
12	Internet Party	4848	0.204083952
13	Democrats for Social Credit	4647	0.195622551
14	Ban1080	4448	0.187245342
15	Independent	3864	0.162660972
16	NZ Independent Coalition	1929	0.081204196
17	Focus New Zealand	1797	0.075647455
18	Money Free Party	391	0.016459741
19	Communist League	135	0.005683031
20	Climate Party	116	0.004883197
21	Human Rights Party	76	0.003199336
22	The Expats	70	0.002946757
23	Alliance	59	0.002483695
24	NZ Economic Euthenics Party	51	0.002146923
25	Patriotic Revolutionary Front	48	0.002020633

(c).

```
cand <- nzge %>%
  filter(voting_type == "Candidate") %>%
```

```

count(election_year, party, wt = votes, name = "votes") %>%
slice_max(votes, n = 1, by = election_year) %>%
transmute(election_year, Candidate_winner = party)

prty <- nzge %>%
  filter(voting_type == "Party") %>%
  count(election_year, party, wt = votes, name = "votes") %>%
  slice_max(votes, n = 1, by = election_year) %>%
  transmute(election_year, Party_winner = party)

cand %>%
  inner_join(prty, by = "election_year") %>%
  arrange(election_year)

```

	election_year	Candidate_winner	Party_winner
1	2002	Labour Party	Labour Party
2	2005	National Party	Labour Party
3	2008	National Party	National Party
4	2011	National Party	National Party
5	2014	National Party	National Party

Problem 2 - Tidyverse: Tennis

```

# url <- "https://raw.githubusercontent.com/JeffSackmann/tennis_atp/refs/heads/master/atp_ma
#
# download.file(url, destfile = "atp_matches_2019.csv", mode = "wb", quiet = TRUE)

```

```
tennis <- read.csv("atp_matches_2019.csv")
```

(a).

```

tennis %>%
  summarise(tournaments_number = n_distinct(tourney_id))

```

	tournaments_number
1	128

I treat any match whose `tourney_id` starts with 2019- as belonging to the **2019 season**, even if some matches occurred in **2018-12**. Using this season-based definition to count tournaments is generally more consistent with tennis statistics practice.

In 2019, there were **128** ATP tournaments based on distinct `tourney_id` values in the 2019 matches file.

(b).

```
winners <- tennis %>%
  filter(round == "F") %>%
  select(tourney_id, winner_id, winner_name)

titles <- winners %>%
  count(winner_id, winner_name, name = "titles") %>%
  arrange(desc(titles), winner_name)
```

```
titles %>%
  count(titles > 1)
```

```
titles > 1  n
1      FALSE 29
2       TRUE 12
```

```
titles %>% filter(titles == max(titles))
```

```
winner_id  winner_name titles
1    106233  Dominic Thiem      5
2    104925  Novak Djokovic     5
```

12 players won more than one tournament, and the top winners each won **5** tournaments.

(c).

```
tennis %>%
  transmute(diff = w_ace - l_ace) %>%
  drop_na(diff) %>%
  pull(diff) %>%
  stats::t.test(mu = 0, alternative = "two.sided")
```

One Sample t-test

```
data: .  
t = 12.373, df = 2693, p-value < 2.2e-16  
alternative hypothesis: true mean is not equal to 0  
95 percent confidence interval:  
 1.434712 1.975088  
sample estimates:  
mean of x  
 1.7049
```

```
tennis %>%  
  transmute(diff = w_ace - l_ace) %>%  
  drop_na(diff) %>%  
  summarise(  
    n = n(),  
    mean_diff = mean(diff),  
    sd = sd(diff),  
    se = sd / sqrt(n),  
    t = mean_diff / se,  
    df = n - 1,  
    p_two_sided = 2 * (1 - stats::pt(abs(t), df = df)),  
    p_greater = 1 - stats::pt(t, df = df),  
    ci_low_t = mean_diff - stats::qt(0.975, df) * se,  
    ci_high_t = mean_diff + stats::qt(0.975, df) * se  
  )
```

	n	mean_diff	sd	se	t	df	p_two_sided	p_greater
1	2694	1.7049	7.151908	0.1377917	12.37302	2693	0	0

	ci_low_t	ci_high_t
1	1.434712	1.975088

There is strong statistical evidence that winners record more aces than losers. The estimated mean difference is about **1.705 aces per match** (95% CI: **1.435 to 1.975**) with nearly 0 p-value, so winners, on average, usually have more than one aces than losers.

(d).

```

bind_rows(
  tennis %>% transmute(player_id = winner_id, player_name = winner_name, won = 1L),
  tennis %>% transmute(player_id = loser_id, player_name = loser_name, won = 0L)
) %>%
  summarise(
    matches = n(),
    wins = sum(won),
    win_rate = wins / matches,
    .by = c(player_id, player_name)
  ) %>%
  filter(matches >= 5) %>%
  filter(win_rate == max(win_rate)) %>%
  arrange(desc(win_rate), desc(matches), player_name)

```

	player_id	player_name	matches	wins	win_rate
1	104745	Rafael Nadal	69	60	0.8695652

Among players with **5 matches**, the highest win rate is **Rafael Nadal** with **60 wins in 69 matches** — win rate **86.96%**.

Problem 3 - Visualization

```

# url <- "https://raw.githubusercontent.com/nytimes/covid-19-data/refs/heads/master/rolling-
#
# download.file(url, destfile = "us-states.csv", mode = "wb", quiet = TRUE)

```

```

# df_us <- read_csv("us.csv", show_col_types = FALSE)
df_states <- read_csv("us-states.csv", show_col_types = FALSE)

```

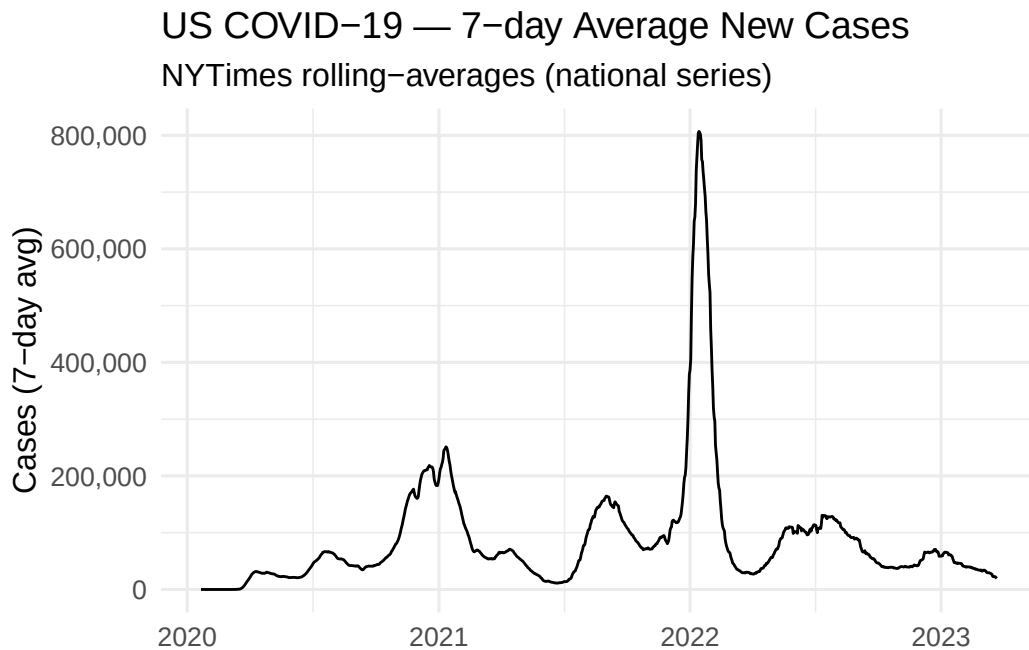
(a).

```

df_us <- df_states %>%
  mutate(date = as.Date(date)) %>%
  group_by(date) %>%
  summarise(cases_avg = sum(cases_avg, na.rm = TRUE), .groups = "drop")

```

```
ggplot(df_us, aes(date, cases_avg)) +
  geom_line(linewidth = 0.5) +
  scale_y_continuous(labels = label_comma()) +
  labs(title = "US COVID-19 - 7-day Average New Cases",
       subtitle = "NYTimes rolling-averages (national series)",
       x = NULL, y = "Cases (7-day avg)") +
  theme_minimal(base_size = 12)
```



Since daily counts may jump around and might be noisy, we could use 7-day avg to shows the real trend in recent periods.

The plot shows a **single major spike at the beginning of 2022**. In addition, there are roughly **five minor spikes**—around mid-2020, late-2020 to early-2021, mid-to-late 2021, mid-2022, and late-2022 to early-2023.

(b).

```
ranked <- df_states %>%
  summarise(
    overall_rate = mean(cases_avg_per_100k, na.rm = TRUE),
    .by = state
```



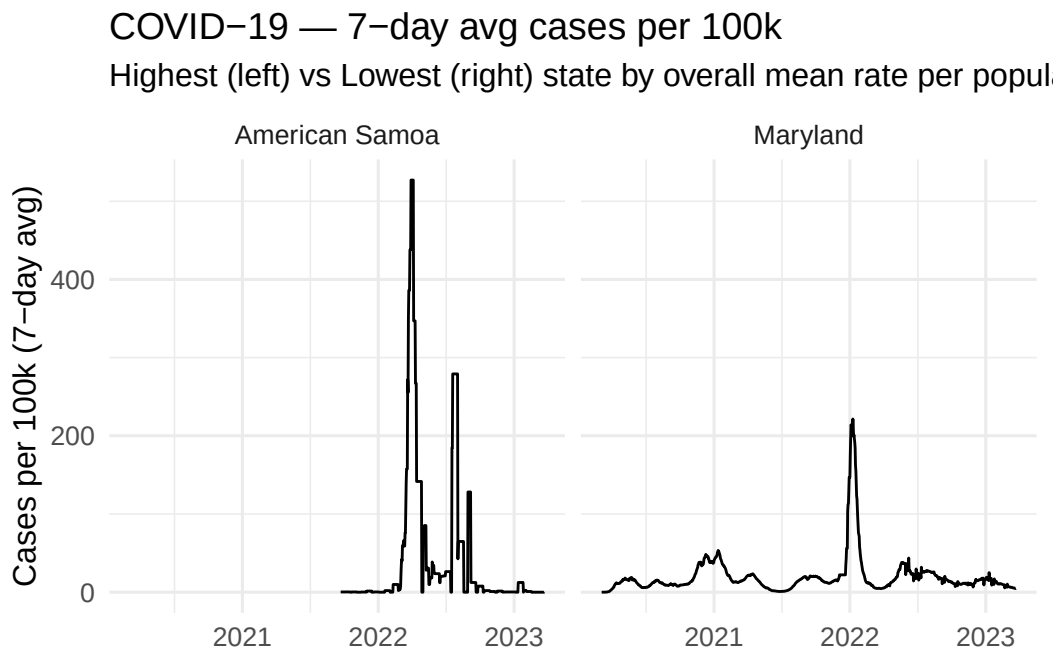
```

) %>%
  arrange(desc(overall_rate))

top_state <- ranked %>% slice_head(n = 1)
bot_state <- ranked %>% slice_tail(n = 1)

df_states %>%
  filter(state %in% c(top_state, bot_state)) %>%
  ggplot(aes(date, cases_avg_per_100k)) +
  geom_line(linewidth = 0.5) +
  facet_wrap(~ state, ncol = 3) +
  scale_y_continuous(labels = label_number(accuracy = 1)) +
  labs(
    title = "COVID-19 - 7-day avg cases per 100k",
    subtitle = paste0("Highest (left) ", "vs Lowest (right) ",
                      , "state by overall mean rate per population"),
    x = NULL, y = "Cases per 100k (7-day avg)"
  ) +
  theme_minimal(base_size = 12)

```



American Samoa has highest overall mean rate and Maryland has lowest.
 American Samoa shows **short, very high spikes** (peaks >500 per 100k) with long stretches

near zero between waves, whereas Maryland's peaks are **much lower** (about 220 per 100k) but **broad and more persistent**, with a higher baseline and ongoing mid-sized waves. The peak of American Samoa is between 2022~2023 and the peak of Maryland is around winter 2021.

(c).

```
thr <- 10
kday <- 7

first_hit <- df_states %>%
  arrange(state, date) %>%
  group_by(state) %>%
  mutate(
    hit = cases_avg_per_100k >= thr,
    hit_k = slide_lgl(hit, ~ all(.x), .before = kday - 1, .complete = TRUE)
  ) %>%
  summarise(first_date = min(date[hit_k], na.rm = TRUE), .groups = "drop") %>%
  filter(is.finite(first_date))

ref_date <- min(first_hit$first_date, na.rm = TRUE)
rank_df <- first_hit %>%
  mutate(days_since_earliest = as.integer(first_date - ref_date)) %>%
  arrange(first_date)
```

```
rank_df$state[1]
```

```
[1] "New York"
```

```
rank_df$first_date[1]
```

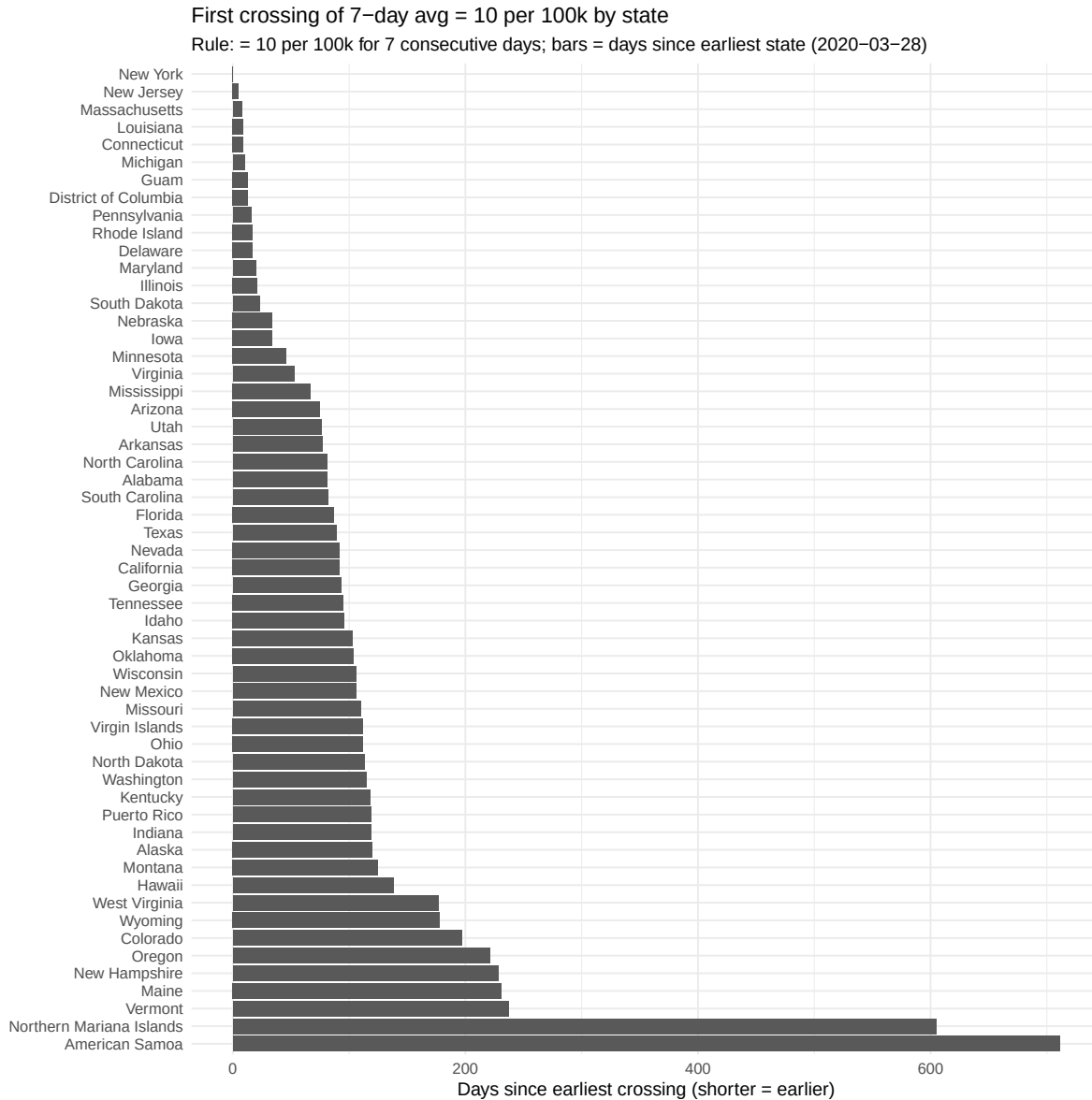
```
[1] "2020-03-28"
```

I defined an “outbreak” as a 7-day average of **10 cases per 100k** sustained for **7 consecutive days** (cases_avg_per_100k). For each state, I identified the **first date** that meets this criterion, then used the **earliest state’s** date as a baseline to compute the **days_since_earliest** for all other states. The earliest state is **New York** at **2020-03-28**.

```

ggplot(rank_df, aes(x = reorder(state, -days_since_earliest)
                    , y = days_since_earliest)) +
  geom_col() +
  coord_flip() +
  scale_y_continuous(labels = label_number(accuracy = 1)) +
  labs(
    title = "First crossing of 7-day avg 10 per 100k by state",
    subtitle = paste0("Rule: ", thr, " per 100k for ", kday,
                      " consecutive days; bars = days since earliest state (",
                      ref_date, ")"),
    x = NULL, y = "Days since earliest crossing (shorter = earlier)"
  ) +
  theme_minimal(base_size = 12)

```



The plotted shows a bar chart where **shorter bars indicate earlier onset**. The top five states are: **New York, New Jersey, Massachusetts, Connecticut and Louisiana.**